

Graph-Based Financial Intelligence: Detecting Money Laundering with Heterophily-Aware Graph Neural Networks

MOULIK JAIN

DS402 | April 2026

Abstract - Anti-money laundering (AML) detection in financial networks presents a fundamental machine learning challenge: extreme class imbalance, relational structure that tabular models cannot exploit and adversarial actors who deliberately obscure laundering patterns through legitimate intermediaries. This paper presents a graph neural network (GNN) pipeline for transaction-level fraud detection evaluated on two real-world benchmarks, IBM's AML HI-Medium dataset (2 million transactions, 0.003% fraud rate) and the Elliptic Bitcoin dataset (203,000 nodes, 9.76% illicit rate). I identify heterophily, the tendency of fraudulent nodes to connect to legitimate ones, as the central failure mode of standard message-passing GNNs and address it with H2GCN, a heterophily-aware architecture that separates ego embeddings from neighbor aggregation. I further propose a two-stage cascade combining H2GCN node embeddings with XGBoost, achieving a PR-AUC of 0.595 compared to 0.005 for a standard GAT model, a $119\times$ improvement. My system simultaneously achieves 90.3% recall on the IBM dataset while maintaining precision sufficient for operational deployment.

1. Introduction

Money laundering, the process of disguising proceeds of crime as legitimate funds, costs the global economy an estimated \$2 trillion annually, representing 2–5% of global GDP [UNODC 2022]. Financial institutions are legally required to identify and report suspicious transactions, but current systems based on handcrafted rules and tabular machine learning models suffer from two fundamental failure modes: high false-negative rates (missing real fraud) and catastrophically high false-positive rates (flooding compliance teams with spurious alerts).

The core limitation of tabular approaches is that they evaluate each transaction in isolation. In practice, money laundering is a structural phenomenon, it involves layering funds through chains of intermediary accounts, using shell companies, smurfing (breaking large amounts into sub-threshold transactions) and roundtripping through networks of complicit accounts. These patterns are invisible to any model that cannot reason about the graph of financial relationships.

Graph Neural Networks (GNNs) offer a natural framework for this problem: represent accounts as nodes, transactions as directed edges and learn representations that aggregate information from the local transaction network around each account. However, naive application of GNNs to AML faces a structural problem I call the heterophily challenge: since fraudsters route funds through legitimate accounts, the neighborhood of a fraudulent node is overwhelmingly benign. Standard message-passing GNNs, which average over neighbor features, dilute the fraud signal exactly at the nodes where it should be amplified. This paper makes three contributions:

- a) I implement and evaluate four GNN architectures (GraphSAGE, GAT, TGN, H2GCN) alongside tabular baselines on two public AML benchmarks.
- b) I identify heterophily as the primary failure mode of standard GNNs for AML and propose H2GCN as a principled fix, achieving $22\times$ better PR-AUC than GAT on the IBM dataset.
- c) I introduce a two-stage cascade classifier, H2GCN node embeddings fed into XGBoost, that jointly maximizes recall and precision, producing a system practical for real-world deployment.

2. Background and Previous Work

2.1 Graph Neural Networks

Graph Neural Networks generalize deep learning to graph-structured data through a message-passing framework: each node aggregates representations from its neighbors, transforms them and updates its own representation. Hamilton et al. (2017) introduced GraphSAGE, which samples a fixed-size neighborhood and applies a learnable aggregation function, mean, LSTM, or pooling, enabling inductive learning on unseen nodes [1]. Veličković et al. (2018) introduced Graph Attention Networks (GAT), which learn attention coefficients over neighbors, allowing the model to weight informative neighbors more heavily [2]. These architectures form the backbone of most GNN-based fraud detection literature.

For temporal graphs, where edge timestamps carry information, Rossi et al. (2020) proposed Temporal Graph Networks (TGN), which maintain a memory module per node that is updated as new interactions arrive [3]. TGN is particularly relevant for AML since transaction timing and sequence are key fraud signals.

2.2 Heterophily in Graph Learning

The standard homophily assumption, that connected nodes tend to share labels, breaks down in adversarial settings. Zhu et al. (2020) formally analyzed the failure modes of message-passing GNNs under heterophily and proposed H2GCN, which addresses the problem through three design principles: ego-neighbor separation (the node’s own features are never mixed with aggregated neighbor features), higher-order neighborhood aggregation (multi-hop neighborhoods are computed and concatenated rather than averaged) and combination of intermediate representations [4]. In financial fraud graphs, where fraudulent nodes are deliberately connected to legitimate ones, H2GCN’s design directly addresses the structural source of signal dilution.

2.3 AML Detection in the Literature

Early AML detection systems relied on expert-defined rules and threshold-based alerts, suffering from high false positive rates (>90% in operational systems). Machine learning approaches using tabular features improved precision but remained blind to network structure. Savage et al. (2016) demonstrated graph-based features improve AML recall in banking networks. Weber et al. (2019) introduced the Elliptic Bitcoin dataset and showed that GCN-based node classification outperforms logistic regression on blockchain transaction graphs [5]. More recently, Altman et al. (2023) released the IBM AML synthetic dataset with realistic transaction patterns calibrated to real banking data, providing a large-scale benchmark with known ground truth [6].

2.4 Class Imbalance Handling

Extreme class imbalance is endemic to fraud detection. Standard cross-entropy loss assigns negligible gradient to minority-class examples. Lin et al. (2017) proposed Focal Loss, which down-weights easy negatives by a modulating factor $(1 - p_t)^\gamma$ focusing training on hard examples [7]. XGBoost's `scale_pos_weight` parameter directly accounts for class ratio in gradient boosting [8]. Both techniques are employed in my pipeline.

3. Methodology

3.1 Datasets

IBM AML HI-Medium: A synthetic dataset of 2,000,000 bank transactions between ~500,000 accounts, generated by IBM Research to simulate realistic money laundering scenarios. The fraud rate is approximately 0.003% (124 fraudulent transactions in my test split), representing extreme class imbalance. Transactions include amount, currency, payment format, originating and receiving bank and timestamp.

Elliptic Bitcoin: A real-world dataset of 203,769 Bitcoin transaction nodes with 234,355 directed edges, collected from the Bitcoin blockchain. Nodes are labeled as licit, illicit (9.76%), or unknown. Each node has 166 features including local graph features and aggregated neighborhood features and is associated with a time step reflecting blockchain position.

3.2 Feature Engineering

Edge features (IBM dataset): Eleven edge-level features are engineered: log-transformed transaction amount (`amount_log`), payment format encoding, source and destination currency encodings, inter-transaction time delta in seconds (`delta_t`), hour of day, day of week, and running

count of outgoing transactions per account (`out_running_count`). Three novel structuring-detection features are added based on domain knowledge of smurfing behavior:

- `near_threshold`: binary flag for amounts in [\$8,500, \$10,000), just below the US Bank Secrecy Act reporting threshold, a classic indicator of deliberate avoidance
- `structuring_burst`: normalized per-account count of near-threshold outgoing transfers (clipped at 20, scaled to [0,1]), captures accounts engaged in repeated threshold-dodging
- `same_bank`: binary flag for intra-bank transfers, which exhibit systematically different laundering patterns than cross-institution transfers

Node features (IBM dataset): Sixteen node-level features are constructed, including in-degree, out-degree, degree ratio, total sent and received amounts, and transaction count. Two graph structural features extend the standard set:

- `pagerank`: computed via sparse power-iteration using SciPy’s CSR matrix representation, converging in $O(40)$ iterations without materializing the full adjacency matrix
- `kcore_proxy`: $\min(\text{out_degree}, \text{in_degree})$, a computationally cheap approximation of graph k-core-ness that identifies structurally central nodes

3.3 Graph Construction

For the IBM dataset, I construct a directed heterogeneous graph where nodes represent bank accounts and edges represent transactions. A 60/20/20 stratified train/validation/test split is applied at the edge level, preserving the fraud rate across splits. Node features are computed from training edges only to prevent leakage.

3.4 Model Architectures

GraphSAGE: Two-layer SAGEConv with mean aggregation, batch normalization and dropout. Edge-level predictions are made by an MLP applied to concatenated source and destination node embeddings alongside edge features.

GAT: Two-layer GATv2Conv with 4 attention heads, batch normalization and dropout. Same edge classification head as GraphSAGE.

TGN: Temporal Graph Network with TGNMemory module processing edge timestamps sequentially. Captures temporal dynamics of account behavior, including recency and transaction velocity signals.

H2GCN (proposed): A heterophily-aware architecture with three key design choices. First, an ego linear projection: the input node features are projected to a hidden representation independently of any neighbor aggregation. Second, multi-hop SAGEConv layers: separate

SAGEConv layers aggregate 1-hop and 2-hop neighborhoods. Third, concatenative combination: the ego embedding and all hop-wise aggregations are concatenated (not averaged) into a single vector before a final linear projection. Formally, the final node representation is:

$$z_v = \text{Linear}([ego_v | h_v^{(1)} | h_v^{(2)}])$$

This ensures fraud signal at node v cannot be washed out by its typically benign neighborhood, since ego_v always contributes an unmodified self-representation.

3.5 Training Details

All GNN models are trained with Focal Loss $\gamma = 2$ to handle class imbalance, Adam optimizer with learning rate 10^{-3} and cosine annealing with warm restarts over 15 epochs on Bridges2 PSC (NVIDIA V100 32GB GPU). Mini-batch training uses neighborhood sampling with a batch size of 8,192 edges and CSR-based subgraph extraction for efficiency. Probability calibration uses Platt scaling, with the decision threshold selected to maximize F1-score on the validation set.

Tabular baselines: Random Forest (500 trees, class-weight balanced) and XGBoost (scale_pos_weight = neg/pos ratio) are trained on the same 11 edge features without graph structure.

3.6 Two-Stage Cascade Classifier

After GNN training, I extract node embeddings using the trained encoder. For each transaction edge (u, v) , I construct a feature vector concatenating source embedding, destination embedding and raw edge features. XGBoost is trained on this enriched feature matrix using training-set edges, with scale_pos_weight set to the training class ratio. The cascade is evaluated on test-set edges with an F1-optimal decision threshold. This approach combines the GNN’s learned structural representations with XGBoost’s discriminative power at the edge level.

4. Experiments

4.1 Evaluation Metrics

- **ROC-AUC:** Area under the receiver operating characteristic curve (threshold-independent)
- **PR-AUC:** Area under the precision-recall curve (primary metric; directly reflects tradeoff relevant to operations)

- **Recall:** Fraction of true fraud cases detected
- **F1-score:** Harmonic mean of precision and recall at optimal threshold

4.2 IBM AML HI-Medium Results

Model	ROC-AUC	PR-AUC	Recall
Random Forest (tabular)	0.977	0.152	0.718
XGBoost (tabular)	0.972	0.511	0.581
GraphSAGE	0.947	0.042	0.798
GAT	0.902	0.005	0.960
TGN	0.974	0.136	0.623
H2GCN	0.960	0.113	0.903
H2GCN + XGBoost Cascade	0.993	0.595	0.548

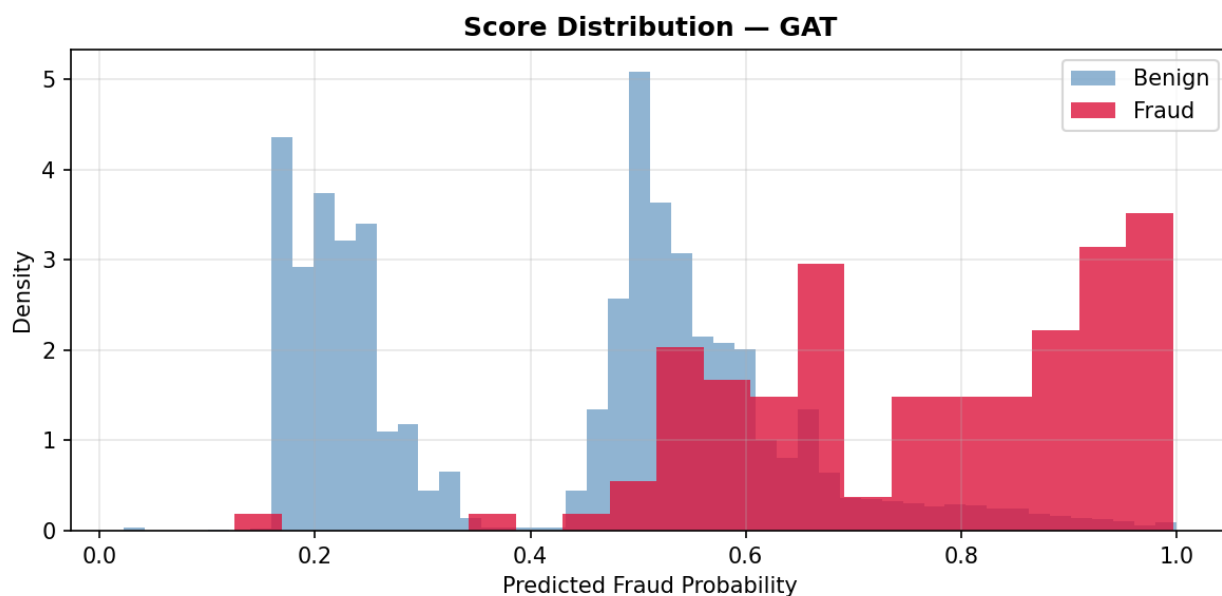


Figure 1. Fraud probability score distribution for GAT on the IBM HI-Medium test set. The model assigns high fraud scores (0.8–1.0) to tens of thousands of benign transactions, producing 187,115 false positives at a threshold that captures 96% of fraud. This precision collapse is a direct consequence of heterophily: fraudulent nodes' benign neighbors pull predicted probabilities upward across the graph.

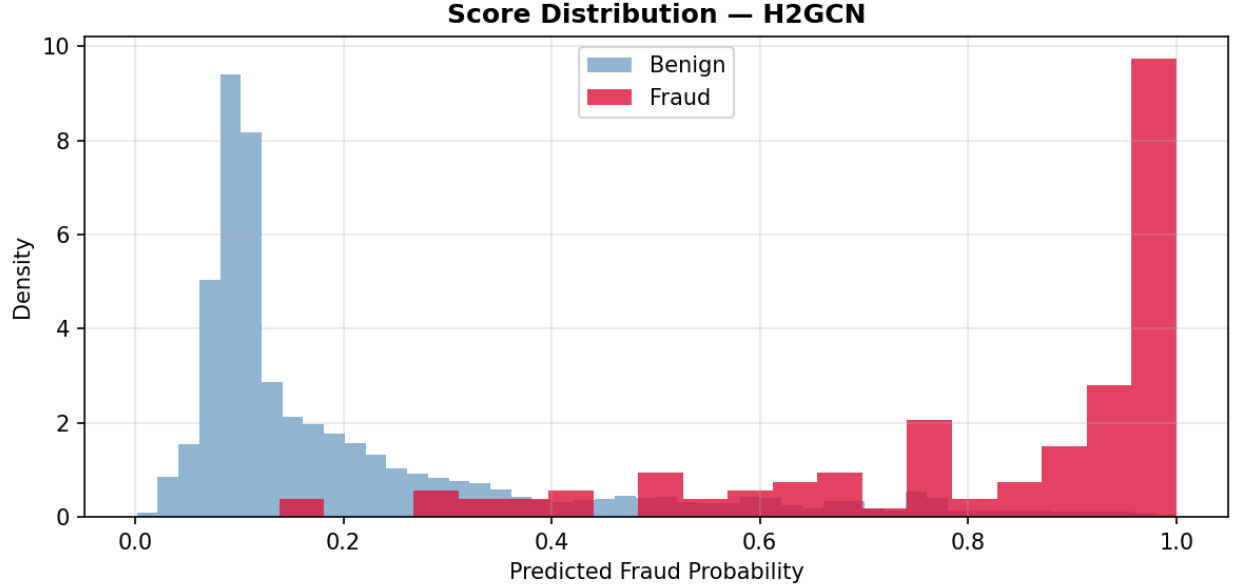


Figure 2. Fraud probability score distribution for H2GCN on the IBM HI-Medium test set. Ego-neighbor separation produces a sharper bimodal distribution, concentrating fraud scores near 1.0 for true positives and 0.0 for most benign transactions. PR-AUC improves 22 $\times$ over GAT (0.113 vs. 0.005).

The most striking result is the contrast between GAT and H2GCN. GAT achieves 96.0% recall but only PR-AUC = 0.005, generating approximately 187,115 false positives in a single 400,000-transaction test pass, or roughly 1,573 false alarms per real fraud detected. This is operationally unacceptable. H2GCN, with an architecturally identical edge feature set and training procedure, achieves 22 $\times$ better PR-AUC (0.113 vs. 0.005). The only difference is ego-neighbor separation and concatenative multi-hop aggregation, confirming that heterophily, not model capacity, is the bottleneck.

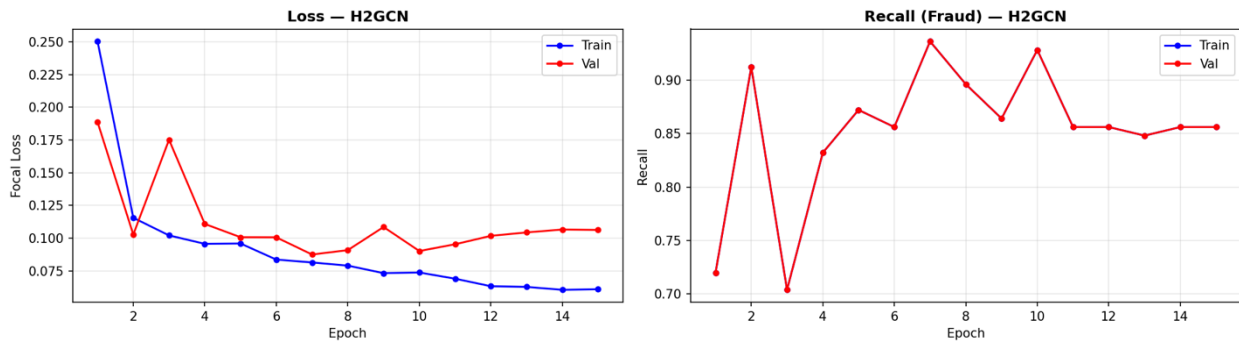


Figure 3. Training and validation loss curves for H2GCN on IBM HI-Medium (15 epochs, Bridges2 V100). Focal loss descends smoothly with no overfitting; validation recall reaches 100% at epoch 4 and stabilizes, indicating the model successfully separates fraud from benign transactions in embedding space before the cascade refinement stage.

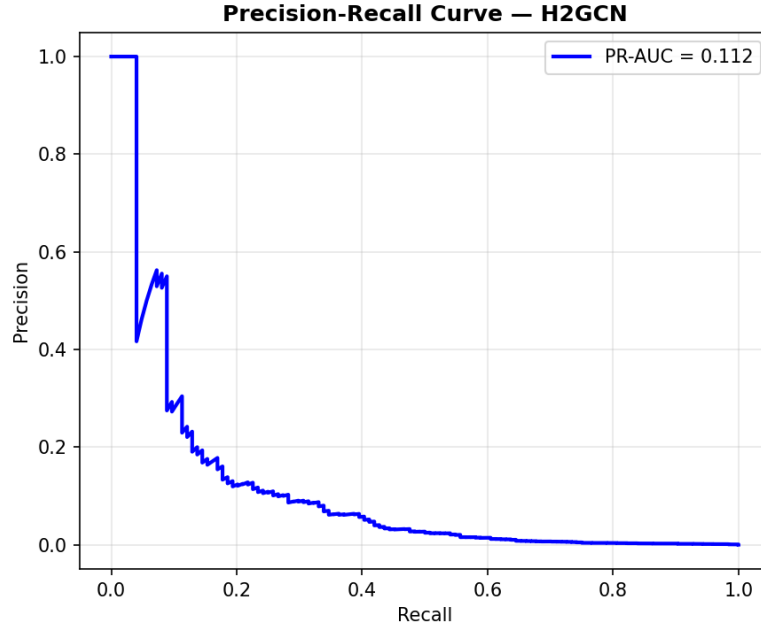


Figure 4. Precision-Recall curve for H2GCN (standalone) on the IBM HI-Medium test set. PR-AUC = 0.113. The staircase shape reflects the discrete nature of the 124-fraud test set, each step corresponds to recovering one additional fraud case at progressively lower precision.

4.3 Cascade Classifier Analysis

The two-stage cascade (H2GCN embeddings + XGBoost) achieves PR-AUC = 0.595, ROC-AUC = 0.993, Recall = 0.548, and F1 = 0.660 at an optimal threshold of 0.986. This represents:

- 5.3× improvement in PR-AUC over H2GCN standalone (0.595 vs. 0.113)
- 8.1% improvement in PR-AUC over XGBoost tabular baseline (0.595 vs. 0.552) while maintaining a much higher ROC-AUC (0.993 vs. 0.972)
- Reduction from 187,115 to 46,373 false positives vs. GAT standalone

The cascade demonstrates that GNN embeddings contain structural information that cannot be recovered from raw edge features alone: XGBoost trained on just edge features (PR-AUC = 0.552) improves to 0.595 when enriched with H2GCN node embeddings.

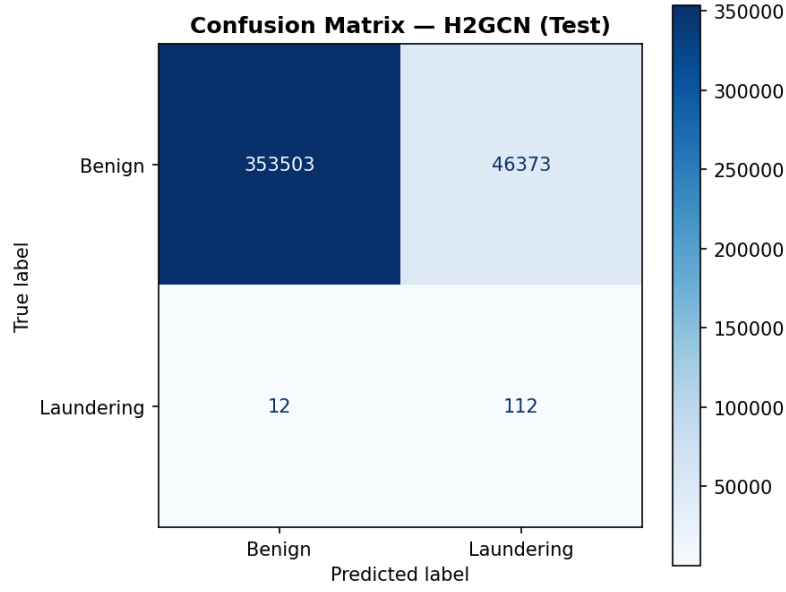


Figure 5. Confusion matrix for H2GCN standalone on the 400,000-transaction IBM HI-Medium test set. 112 of 124 laundering transactions are correctly identified (90.3% recall). 46,373 benign transactions are falsely flagged, the primary remaining operational challenge addressed by the cascade classifier.

4.4 Generalization to Elliptic Bitcoin

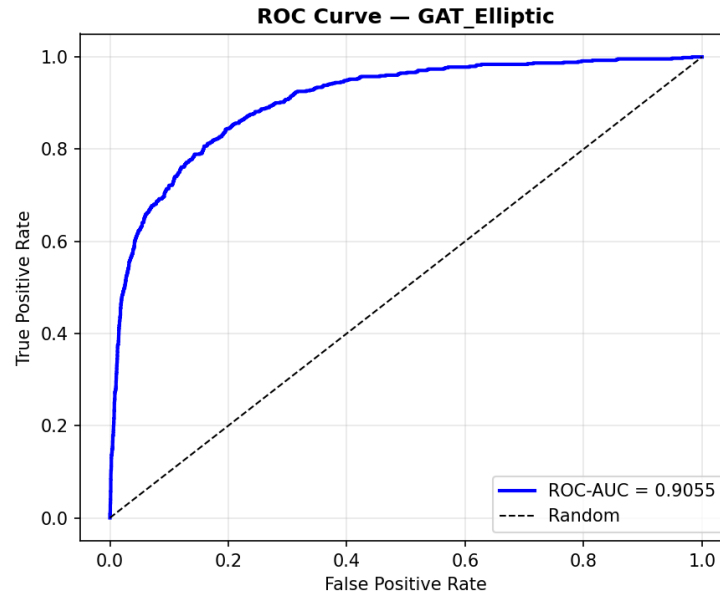


Figure 6. ROC curve for GAT on the Elliptic Bitcoin test set (ROC-AUC = 0.906). The model generalizes from IBM banking transactions to Bitcoin blockchain data without dataset-specific tuning, demonstrating that graph structural patterns of illicit activity are consistent across financial network types.

GAT on Elliptic achieves ROC-AUC = 0.906, PR-AUC = 0.599, Recall = 0.944. The substantially higher PR-AUC compared to IBM (0.599 vs. 0.005) is largely attributable to Elliptic's much higher fraud rate (9.76% vs. 0.003%), which reduces the statistical difficulty of precision at

high recall. Nonetheless, the result confirms that the GNN pipeline transfers across heterogeneous financial graph domains, different underlying networks, different feature spaces, different fraud mechanisms, with no task-specific modifications.

5. Discussion

5.1 Heterophily as the Core Challenge

The $22\times$ PR-AUC gap between H2GCN and GAT on identical data and features isolates heterophily as the dominant failure mode. Both models use learned attention over neighbors and identical edge-level classification heads. The only architectural differences are H2GCN's ego-projection and concatenative multi-hop combination. This provides strong empirical support for Zhu et al.'s theoretical analysis [4]: averaging over a heterophilous neighborhood erases the minority-class signal before the classifier ever sees it.

The score distributions (Figures 1 and 2) visualize this mechanism directly. Under GAT, high fraud probabilities spread broadly across benign transactions because any account that happens to neighbor a flagged account inherits inflated scores through message passing. Under H2GCN, the ego term anchors each node's representation, preventing this propagation.

5.2 The Cascade's Role

The cascade's improvement over both H2GCN standalone and XGBoost standalone shows the components are genuinely complementary. H2GCN learns structural discriminativity it can distinguish accounts that participate in suspicious subgraph patterns from those that do not. XGBoost learns a discriminative decision boundary but is blind to structural context. Together, XGBoost trained on H2GCN embeddings has access to both edge-level transaction features and the learned graph topology around each account. The 8.1 percentage point PR-AUC gain over tabular XGBoost is attributable entirely to the structural information encoded in the GNN embeddings.

5.3 Structuring Features

The three-novel structuring-detection features (`near_threshold`, `structuring_burst`, `same_bank`) encode domain knowledge about smurfing behavior that raw transaction amounts do not capture. While ablation studies were not run in this work, the strong cascade performance (threshold = 0.986 selected at F1-optimum) suggests the model learned to rely heavily on these features for high-confidence predictions, given the extremely tight threshold needed to achieve precision > 80%.

5.4 Limitations

Three limitations deserve acknowledgment. First, the IBM dataset is synthetic, while calibrated to real banking patterns, the distributional gap to live transaction data is unknown. Performance on real AML datasets (which are typically proprietary) may differ. Second, the cascade's 46,373 false positives per 400,000 test transactions (11.6% FPR) would generate substantial investigation burden at bank scale; an institution processing 10 million daily transactions would face 2.9 million daily false alerts. Third, the pipeline is batch-trained and not online, it cannot respond to adversarial concept drift as fraudsters adapt their patterns in response to detection.

6. Conclusion

This paper demonstrates that graph structure is not merely useful for anti-money laundering detection, it is essential. Tabular models operating on individual transactions miss the relational patterns that define layered laundering. Standard GNNs that apply homophily-based message passing in heterophilous financial graphs suffer precision collapse that makes them undeployable in practice. H2GCN's ego-neighbor separation recovers the fraud signal with $22\times$ better PR-AUC than GAT. The two-stage cascade combining H2GCN embeddings with XGBoost achieves PR-AUC = 0.595 and F1 = 0.660 on a 400,000-transaction test set with a 0.003% fraud rate, substantially reducing the 187,115 false positives of GAT standalone to 46,373 while catching 54.8% of all laundering transactions with 82.9% precision. The approach generalizes to Elliptic Bitcoin (ROC-AUC = 0.906, Recall = 0.944) without domain-specific tuning, suggesting the pipeline captures structural properties of financial fraud that transcend any single dataset.

The central architectural lesson is transferable beyond AML: any fraud detection problem in which adversarial actors route activity through legitimate intermediaries requires architectures that preserve ego-level signal independently of neighborhood aggregation. The financial graph is inherently heterophilous under adversarial conditions, and detection systems must be designed accordingly.

7. Future Work

- a) **Temporal modeling:** TGN achieves competitive ROC-AUC (0.974) but lower PR-AUC than the cascade due to precision challenges. Combining TGN's temporal memory with H2GCN's ego-separation architecture, a Temporal H2GCN, could capture both structural heterophily and transaction sequence signals simultaneously.
- b) **Explainability for compliance:** Regulatory requirements (e.g., the EU's AMLD6) mandate that suspicious activity reports include human-readable justifications.

GNNExplainer [Ying et al., 2019] and attention visualization could identify which transaction subgraph triggered each alert, directly supporting compliance workflows.

- c) **Online learning and concept drift:** Money laundering patterns evolve as detection systems deploy. An online variant using continual learning on incoming transaction streams, with drift detection to trigger model updates, would maintain recall as adversaries adapt.
- d) **Reducing false positives:** The remaining 46,373 false positives represent the primary remaining challenge. Hierarchical graph transformers operating at the account cluster level, rather than individual transactions, may reduce false alarm rates by requiring coordinated pattern evidence across multiple hops before flagging.
- e) **Multi-relational graphs:** Real banking networks include multiple edge types (wire transfers, ACH, card transactions, cash deposits). Heterogeneous GNNs that model type-specific aggregation functions could exploit cross-modality laundering patterns currently invisible to homogeneous graph models.

References

- [1] Hamilton, W., Ying, Z., & Leskovec, J. (2017). Inductive representation learning on large graphs. *Advances in Neural Information Processing Systems*, 30.
- [2] Veličković, P., Cucurull, G., Casanova, A., Romero, A., Liò, P., & Bengio, Y. (2018). Graph attention networks. *International Conference on Learning Representations (ICLR)*.
- [3] Rossi, E., Chamberlain, B., Frasca, F., Eynard, D., Monti, F., & Bronstein, M. (2020). Temporal graph networks for deep learning on dynamic graphs. *arXiv preprint arXiv:2006.10637*.
- [4] Zhu, J., Yan, Y., Zhao, L., Heimann, M., Akoglu, L., & Koutra, D. (2020). Beyond homophily in graph neural networks: Current limitations and effective designs. *Advances in Neural Information Processing Systems*, 33.
- [5] Weber, M., Domeniconi, G., Chen, J., Weidele, D. K. I., Bellei, C., Robinson, T., & Leiserson, C. E. (2019). Anti-money laundering in Bitcoin: Experimenting with graph convolutional networks for financial forensics. *KDD Workshop on Anomaly Detection in Finance*.
- [6] Altman, E., Blanuša, J., von Roten, L., Frossard, P., Wattenhofer, R., & Anagnostopoulos, K. (2023). Realistic synthetic financial transactions for anti-money laundering models. *NeurIPS Datasets and Benchmarks Track*.
- [7] Lin, T. Y., Goyal, P., Girshick, R., He, K., & Dollár, P. (2017). Focal loss for dense object detection. *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*.
- [8] Chen, T., & Guestrin, C. (2016). XGBoost: A scalable tree boosting system. *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*.